

· [KLDP.org](#) · [KLDP Wiki](#) · [KLDP.net](#) · [통합 검색](#) ·



[Subversion Book/Developer Information](#)



[FrontPage](#) | [FindPage](#) | [TitleIndex](#) | [RecentChanges](#) |

[SubversionBook/GuidedTour](#) › [SubversionBook/BasicConcepts](#) ›
[SubversionBook/BranchingAndMerging](#) ›
[SubversionBook/RepositoryAdministration](#) › [SubversionBook/AdvancedTopics](#) ›
[SubversionBook/DeveloperInformation](#)

Login:

Password:

[Join](#)

You are going to have a new love affair.

[HelpOnMacros](#)
[wafe](#)
[GNULibrary/fputc](#)
[DocbookSgml/Snort-Statistics-HOWTO](#)

1Chapter. 개발자 정보

Table of Contents

- 1.1.
- 1.1.1. [계층화 된 프로그램 라이브러리 설계](#)
 - 1.1.1.1. [저장소\(repository\)층](#)
 - 1.1.1.2. [저장소\(repository\) 액세스층](#)
 - 1.1.2.1. [RA-DAV \(HTTP/DAV를 사용한 저장소\(repository\) 액세스\)](#)
 - 1.1.2.2. [RA-SVN \(코유의 프로토콜에 의한 저장소\(repository\) 액세스\)](#)
 - 1.1.2.3. [RA-Local \(저장소\(repository\)에의 직접적인 액세스\)](#)
 - 1.1.2.4. [Your RA Library Here](#)
 - 1.1.1.3. [클라이언트층](#)
- 1.2. [API의 이용](#)
 - 1.2.1. [Apache Portable Runtime 프로그램 라이브러리](#)
 - 1.2.2. [URL 와 Path 의 요구](#)
 - 1.2.3. [C 와 C++ 이외의 언어의 이용](#)
- 1.3. [작업 카피 관리 area의 내부](#)
 - 1.3.1. [Entries 파일](#)
 - 1.3.2. [수정원카피와 속성 파일](#)
- 1.4. [WebDAV](#)
- 1.5. [메모리프를 사용한 프로그래밍](#)
- 1.6. [Subversion에의 공헌](#)
 - 1.6.1. [커뮤니티에의 참가](#)
 - 1.6.2. [원시 코드의 취득](#)
 - 1.6.3. [커뮤니티의 방식에 정통하는 것](#)
 - 1.6.4. [코드의 변경과 테스트](#)
 - 1.6.5. [변경점의 제공](#)



kss@kldp.org

1.1.

Subversion 는 open source의 소프트웨어 프로젝트로, Apache 스타일의 소프트웨어 라이선스를 가지고 있습니다. 프로젝트는 캘리포니아에 본거지가 있는 소프트웨어 개발 회사 CollabNet, Inc., 의 경제적인 지원을 받고 있습니다. 이 커뮤니티는 Subversion의 개발을 둘러싸고 구성되어 있습니다만, 이 프로젝트에 시간을 할애해 주거나 주의를 향하여 받을 수 있는 것 같은 형태로 무상 원조해 주는 사람을 항상 환영해 있습니다. 자원봉사는 어떤 형태의 원조도 할 수가 있습니다. 그것은, 버그를 찾아내거나 테스트하거나 벌써 있는 코드를 개량하거나 완전히 새로운 기능을 추가하거나라고 한 것을 포함합니다.

이 장은 원시 코드에 자신의 손을 실제로 붙히는 것에 의해 Subversion 의 지금 확실히 일어나고 있는 진화를 원호하려고 하는 사람을 향한 것입니다. 소프트웨어의 것 좀 더 상세하게 접해 Subversion 자신을 개발하는데 혹은, Subversion 프로그램 라이브러리를 사용한 완전하게 새로운 툴을 쓰기 위해서(때문에) 필요하게 되는 기술적으로 중요한 점에 대해 설명합니다. 만약, 그런 레벨의 이야기에 참가하고 싶지 않은 것이면, 이 아키라는 파견해 주어 상당히 입니다. Subversion의 유저로서의 경험에는 영향을 주지 않으므로.

1.1. 계층화 된 프로그램 라이브러리 설계

Subversion은 모듈화된 설계가 되어 있어, C프로그램 라이브러리의 모임과 해 실장되고 있습니다. 프로그램 라이브러리의 각각은 자주(잘) 정의된 목적과 인터페이스 (을)를 가지고 있어, 대부분의 모듈은 세 개의 주요한 층의 어떤 것인가에 속합니다. 저장소(repository)층, 저장소(repository) 액세스층(RA), 그리고 클라이언트층입니다. 이러한 층에 도착해 간단하게 보고 갑니다만, 최초로 Table 7-1 에 있는 Subversion 프로그램 라이브러리 일람을 봐 주세요. 일관한 논의와 하기 위한(해), 프로그램 라이브러리는, 확장자(extension)를 들여다 본 Unix 의 프로그램 라이브러리 명칭으로 참조하기로 하겠습니다 (예를 들어: libsvn_fs, libsvn_wc, mod_dav_svn).

Table 1-1. Subversion 프로그램 라이브러리의 일람

Table 7-1 에 "다양한"라는 말이 하나만 나와 있다는 것은, 좋은 설계인 증거입니다. Subversion 개발 팀은 각각의 기능이, 올바르게 층과 프로그램 라이브러리에 있는 것을 확인하는 것을, 중요한 일이라고 생각하고 있습니다. 아마, 모듈화한 설계의 1번 큰 이점은 개발자의 관점으로부터 본 복잡함 (을)를 줄일 수가 있는 것입니다. 개발자로서 아선반은 곧바로, "이야기의 개요"를 알 수가 있어 거기에 따라 비교적 간단하게 어느 특정의 기능의 장소를 특정 할 수가 있게 됩니다.

그리고, 실제로 단지 큰 그림을 그리는 것보다도, 개발자에게 "이야기의 개요"를 전하는 도움이 되는 것은 무엇일까요? Subversion 프로그램 라이브러리가 어떻게 서로 제휴하고 있는지를 이해하는 도움으로서 Subversion의 계층도, Figure 7-1 를 봐 주세요. 프로그램은(유저에 의해 기동된 뒤) 이 그림 위로부터 시작되어, 아래로 향해 흐릅니다.

Figure 1-1. Subversion 의 개요

모듈화외의 이점은, 주어진 모듈을 코드의 다른 부분에 영향 주는 것 없이, 같은 API를 실장한 새로운 프로그램 라이브러리로 옮겨놓는 것이 할 수 있다고 하는 것입니다. 어떤 의미로 이것은 Subversion 내부에서 벌써 일어나 있는 것입니다. libsvn_ra_dav, libsvn_ra_local, 그리고 libsvn_ra_svn 의 모든 것은, 같은 인터페이스를 실장하고 있습니다. 그리고, 이 3개(살) 모두 하지만, 저장소(repository)층과 교환합니다. libsvn_ra_dav 와 libsvn_ra_svn 는 네트워크 넘어로 그렇게 하고, libsvn_ra_local 는 직접 저장소(repository)에 접속합니다.

클라이언트 자신도 또 Subversion의 설계에서의 모듈성을 분명히 가리키고 있습니다. Subversion은 현재로서는 커멘드 라인 클라이언트 프로그램만을 실장하고 있습니다만, Subversion을 위해서(때문에) GUI로서 행동한다 같은 서드 파티에 의해 개발되고 있는 몇개의 프로그램이 있습니다. 이러한 GUI도, 벌써 실장되고 있는 커멘드 클라이언트 (와)과 같은 API를 이용합니다. Subversion의 libsvn_client 프로그램 라이브러리는 Subversion 클라이언트를 설계하는데 필요한 대부분의 기능을 위해서(때문에) 이용할 수가 있습니다. (>참조).

1.1.1. 저장소(repository)층

Subversion의 저장소(repository)층을 참조할 때, 일반적으로, 두 개의 프로그램 라이브러리에 대해 말하고 있습니다. 저장소(repository) 프로그램 라이브러리와 파일 시스템 프로그램 라이브러리 입니다. 이러한 프로그램 라이브러리는 버전 관리된 데이터의 다양한 리비전 (을)를 위한 격납과 보고의 구조를 제공하고 있습니다. 이 층은 저장소(repository) 액세스층 (을)를 경유해 클라이언트층과 연결되어 있어, Subversion 이용자로부터 보면(자), "통신의 상대방"에 있는 것으로 보입니다.

Subversion의 파일 시스템은 libsvn_fs API에 의해 액세스 되어 operating system에 인스톨 되고 있는 커널 레벨의 의미에서의 파일 시스템(Linux의 ext2 나, NTFS와 같이)이 아니고, 가상적인 파일 시스템입니다. "파일"과 "디렉토리"를 (자신이 좋아하는 조가비를 사용해 조작할 수가 있는 것 같은) 현실의 파일과 디렉토리로서 보존하는 것이 아니라, 연구 최종 단계의 보존의 구조로서 데이터베이스 시스템을 사용합니다. 현재, 이용할 수 있는 데이터베이스 시스템은 Berkeley DB입니다. [1] 그러나, Subversion의 향후의 릴리스 중(안)에서 개발 커뮤니티에 의해 흥미를 당기고 있는, 다른 연구 최종 단계 데이터베이스 시스템의 이용 가능성이 있습니다. 예를 들어, 오픈 데이터베이스 커넥티비티- (ODBC) 등입니다.

libsvn_fs 가 제공하는 파일 시스템 API는 다른 파일 시스템 API 그렇지만 기대할 수 있는 것 같은 기능을 가지고 있는: 파일이나 디렉토리의 작성이나 삭제를 할 수 있어, 카피나, 이동을 할 수 있어 파일의 내용을 수정할 수가 있어, 등 등입니다. 너무 일반적이지 않는 것 같은 기능도 있습니다. 예를 들어, 파일이나 디렉토리에 부수 했다 메타데이

이타("속성")의 추가, 변경, 삭제, 등입니다. 한층 더 Subversion 파일 시스템은 버전화 가능한 파일 시스템으로, 이것은, 디렉토리 트리로 변경을 더하면(자), Subversion은 그 변경 이전에, 그 트리가 어떻게 보일까를 기억해 둔다고 하는 것을 의미합니다. 그리고, 한층 더 그 전의 변경전, 한층 더 그 전, 등입니다. 이와 같이 해, 파일 시스템에 무엇인가를 최초로 추가했는데까지의 모든 버전으로 돌아올 수가 있습니다.

트리에 가세한 모든 변경은 Subversion의 트랜잭션(transaction) 중(안)에서 실행 됩니다. 이하는, 파일 시스템을 수정하기 위한 단순하고 일반적인 수속입니다:

1. Subversion 트랜잭션(transaction)의 개시
2. 수정의 실행(추가, 삭제, 속성의 수정, 등)
3. 트랜잭션(transaction)의 커밋

트랜잭션(transaction)을 커밋하면(자), 파일 시스템의 변경은 역사상의 사건으로서 영구히 기록됩니다. 이러한 각각의 사이클은 트리에 새로운 리비전을 만들어, 각각의 리비전은 "있는 것이 어느 같았는가"라고 하는 순수한 snapshot로 하고 있고 개에서도 액세스 할 수 있게 됩니다.

트랜잭션(transaction)을 방해 하는 것

Subversion의 트랜잭션(transaction)라고 하는 개념은, 특히, 그것과 매우 가까운 의미의 libsvn_fs 에 있는 데이터베이스의 실제의 코드 (을)를 보여지면(자), 데이터베이스 자신이 서포트하고 있는 트랜잭션(transaction)와 용이하게 혼란해 버립니다. 양쪽 모두, 불가분성과 분리성을 가지고 있습니다. 바꾸어 말하면(자), 트랜잭션(transaction)는 있는 처리의 모임을, "전인가, 무인가" 그렇다고 하는 형태로 실행하는 능력을 줍니다 그 모임안에 있는 모든 처리는, 완전하게 성공하는지, 아무것도 일어나지 않았는지의 같게 다루어 집니다 그리고, 트랜잭션(transaction)중에, 그 데이터에 다른 처리를 하는 프로세스에는 일절 간섭하지 않습니다.

데이터베이스 트랜잭션은 일반적으로, 데이터베이스 자신의 데이터의 수정에 관계한 작은 몇개의 조작을 포함하고 있습니다. (예를 들어, 테이블행의 내용을 수정하는 것 등입니다). Subversion의 트랜잭션(transaction)는, 좀 더 큰 범위의 것으로, 그것은, 파일 시스템 트리의 다음의 리비전으로서 격납되는 것을 목적으로 한, 파일이나 디렉토리에 대한 몇개의 수정을 한다 같은 조작을 포함하고 있습니다. 혼란이 없을 것 같다면, 이렇게 생각해 주세요: Subversion은 Subversion 트랜잭션(transaction)의 생성중에, 데이터베이스 트랜잭션 (을)를 사용합니다(그래서, 만약 Subversion 트랜잭션(transaction)의 생성이 실패하면, 데이터베이스는 최초부터 그 생성을 하지 않는 것처럼 보입니다)

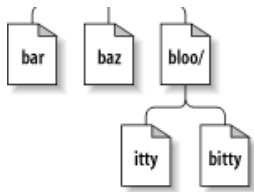
파일 시스템 API 이용자에게 있어 행운의 일로, 데이터베이스 시스템 자신에 의해 준비되어 있는 트랜잭션(transaction) 서포트는, 대부분의 경우 숨어 있어 보이지 않습니다. (보통 모듈화된 프로그램 라이브러리의 설계로부터 기대되는 것입니다만). 파일 시스템 자신의 실장을 개시하는 경우에 처음, 그러한 일이 보이게(혹은 흥미롭게 생각되게) 됩니다.

파일 시스템 인터페이스에 의해 준비되는 기능의 대부분은 파일 시스템 패스에 대해서 일어나는 조작으로서 제공됩니다. 즉, 파일 시스템의 외부로부터는, 파일이나 디렉토리의 개별의 리비전을 기술해, 액세스 하는, 주요 없음 쌍은, /foo/bar (와)과 같은 패스 캐릭터 라인을 사용하는 것을 통해서 제공되어 그것은 정확히, 당신이 정든 조가비 프로그램을 통해서 파일이나 디렉토리에 액세스 한다 같은 기분이 듭니다. 적절한 패스 명분을 세워 있고 API 함수에 건네주는 것에 의해, 새로운 파일이나 디렉토리를 추가할 수가 있습니다. 같은 구조를 사용해 그 파일 등에 관계한 정보를 문의할 수가 있습니다.

대부분의 파일 시스템과는 달라, 패스만을 지정하는 것은, Subversion 의 파일이나 디렉토리를 특정하는데 충분한 정보가 아닙니다. 디렉토리 트리를 이차원의 시스템이라고 생각해 주세요. 여기서, 어느 노드의 형제는, 왼쪽에서 오른쪽으로 이동하는 것을 표현하고 있어, 사브디렉토리에 내려 간다 의는, 아래로 향한 이동이라고 생각해 주세요. Figure 7-2 는 전형적인 트리의 표현을 나타내고 있습니다.

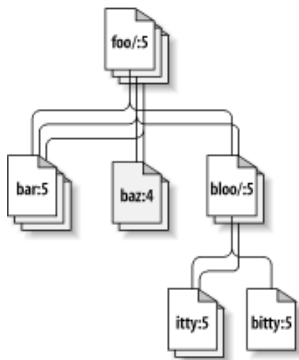
Figure 1-2. 이차원안의 파일과 디렉토리





물론 Subversion의 파일 시스템은 숨은 제 3 차원을 가지고 있습니다만, 그것은 대부분의 파일 시스템이 가지고 있지 않은 것입니다. 그것은 시간의 차원입니다! [2] 파일 시스템 인터페이스로, path 인수를 가지는 거의 모든 함수는 또 root 인수도 지정하지 않으면 안됩니다. 이 `svn_fs_root_t` 인수는, 리비전인가, Subversion의 트랜잭션(transaction)(그것은 보통은 리비전 되어야 할 물건입니다)의 어느 쪽인지를 나타내, 리비전 32의 `/foo/bar` 라고 리비전 98의 같은 패스와의 사이의 차이를 이해하는데 필요하게 되는 삼차원 문맥을 준비합니다. Figure 7-3 는 Subversion 파일 시스템의 우주에 추가된 차원에 대한 리비전 히스토리를 나타내고 있습니다.

Figure 1-3. 리비전 시각제 3 차원!



이전 지적인 것처럼, `libsvn_fs` API 는 다른 파일 시스템과 외관은 자주(잘)에서 있지만, 이 훌륭한 버전 관리 능력만은 예외입니다. 그것은 버전 파일 시스템에 흥미가 있는 모든 프로그래머에 의해 이용할 수 있도록(듯이) 설계되었습니다. 우연의 일치가 아닙니다만, Subversion 자신도 그 기능에 흥미가 있습니다. 그러나, 파일 시스템 API (은)는 기본적인 파일과 디렉토리의 버전 관리를 서포트하고 있습니다만 Subversion은 한층 더 대부분을 요구합니다. 그리고 그것은 `libsvn_repos` 가 제공하는 것입니다.

Subversion 저장소(repository) 프로그램 라이브러리(`libsvn_repos`)는 기본적으로는 파일 시스템 기능의 주위에 있는 래퍼 프로그램 라이브러리입니다. 이 프로그램 라이브러리는 저장소(repository) 레이아웃의 작성, 파일 시스템의 초기화를 올바르게 실행하는 것, 등에 책임을 가집니다. `Libsvn_repos` 는 또 혹은 실장합니다. 특정의 처리가 실행될 때 저장소(repository)의 코드에 의해 실행되는 스크립트입니다. 이 같은 란 통지, 인증, 혹은 저장소(repository) 관리자가 바라는 것 같은 다양한 목적에 있어 도움이 되는 것입니다. 이 타입의 기능과 저장소(repository) 프로그램 라이브러리 에 의해 제공되는 것 외의 유틸리티는 버전화 파일 시스템의 실장에 강하게 관련하고 있는 것은 아닙니다. 그것이 독자적인 프로그램 라이브러리로서 실장된 이유입니다.

`libsvn_repos` API를 사용하려고 하는 개발자에게는, 그것이 파일 시스템 인터페이스에 대한 완전한 래퍼가 아닌 것을 알겠지요. 즉, 파일 시스템 조작의 일반적인 사이클에 있는 주요한 이벤트에 붙어만 저장소(repository) 인터페이스에 의해 랩 됩니다. 그 안의 몇개인가는, Subversion 트랜잭션(transaction)의 생성이나 커밋, 리비전 속성의 수정 등입니다. 이러한 특정의 이벤트는 거기에 관련한 혹은 있기 (위해)때문에, 저장소(repository)층에 의해 랩 됩니다. 장래적으로는 다른 이벤트도 저장소(repository) API에 의해 랩 될지도 알려지지 않습니다. 그러나, 나머지의 파일 시스템의 작용의 모든 것은 `libsvn_fs` API (와)과 함께 직접 실행됩니다.

예를 들어, 디렉토리가 추가되는 파일 시스템의 새롭다 리비전을 만들기 위한, 저장소(repository)와 파일 시스템 인터페이스에 붙어, 사용법을 설명한 코드가 있습니다. 이 예로(그리고, 이 본전체를 통해서 모든 다른 예에서도), `SVN_ERR` 매크로는 단지 랩 한 함수로부터의 실패했을 경우의 에러 코드의 체크입니다. 그리고 그러한 것 (이)가 있으면 그 에러를 돌려줍니다.

Example 1-1. 저장소(repository)층의 이용

```
/* Create a new directory at the path NEW_DIRECTORY in the Subversion
```

```

    repository located at REPOS_PATH.  Perform all memory allocation in
    POOL.  This function will create a new revision for the addition of
    NEW_DIRECTORY.  */
static svn_error_t *
make_new_directory (const char *repos_path,
                    const char *new_directory,
                    apr_pool_t *pool)
{
    svn_error_t *err;
    svn_repos_t *repos;
    svn_fs_t *fs;
    svn_revnum_t youngest_rev;
    svn_fs_txn_t *txn;
    svn_fs_root_t *txn_root;
    const char *conflict_str;

    /* Open the repository located at REPOS_PATH.  */
    SVN_ERR (svn_repos_open (repos, repos_path, pool));

    /* Get a pointer to the filesystem object that is stored in
       REPOS.  */
    fs = svn_repos_fs (repos);

    /* Ask the filesystem to tell us the youngest revision that
       currently exists.  */
    SVN_ERR (svn_fs_youngest_rev (youngest_rev, fs, pool));

    /* Begin a new transaction that is based on YOUNGEST_REV.  We are
       less likely to have our later commit rejected as conflicting if we
       always try to make our changes against a copy of the latest snapshot
       of the filesystem tree.  */
    SVN_ERR (svn_fs_begin_txn (txn, fs, youngest_rev, pool));

    /* Now that we have started a new Subversion transaction, get a root
       object that represents that transaction.  */
    SVN_ERR (svn_fs_txn_root (txn_root, txn, pool));

    /* Create our new directory under the transaction root, at the path
       NEW_DIRECTORY.  */
    SVN_ERR (svn_fs_make_dir (txn_root, new_directory, pool));

    /* Commit the transaction, creating a new revision of the filesystem
       which includes our added directory path.  */
    err = svn_repos_fs_commit_txn (conflict_str, repos,
                                   youngest_rev, txn);
    if (! err)
    {
        /* No error?  Excellent!  Print a brief report of our success.  */
        printf ("Directory '%s' was successfully added as new revision "
               "'%' SVN_REVNUM_T_FMT '". Wn", new_directory, youngest_rev);
    }
    else if (err->apr_err == SVN_ERR_FS_CONFLICT)
    {
        /* Uh-oh.  Our commit failed as the result of a conflict
           (someone else seems to have made changes to the same area
           of the filesystem that we tried to modify).  Print an error
           message.  */
        printf ("A conflict occurred at path '%s' while attempting "
               "to add directory '%s' to the repository at '%s'. Wn",
               conflict_str, new_directory, repos_path);
    }
    else
    {
        /* Some other error has occurred.  Print an error message.  */
        printf ("An error occurred while attempting to add directory '%s' "
               "to the repository at '%s'. Wn",

```

```

        new_directory, repos_path);
    }

    /* Return the result of the attempted commit to our caller.  */
    return err;
}

```

이 코드로, 저장소(repository)와 파일 시스템 인터페이스의 양쪽 모두에 대하는 호출이 있습니다. `svn_fs_commit_txn`를 사용해 트랜잭션(transaction)를 간단하게 커밋할 수 있습니다. 그러나, 파일 시스템 API는 저장소(repository) 프로그램 라이브러리의 혹은 구조에 대해서는 아무것도 모릅니다. 만약 Subversion 저장소(repository)에 트랜잭션(transaction)를 커밋 할 때마다 자동적으로 어떤 종류의 비Subversion적인 작업을 실행시키고 싶은 경우, (예를 들어, 개발자 메일링리스트에 그 트랜잭션(transaction)로 일어난 모두 변경을 설명하는 email 을 송신하는, 등), 그 함수의 `libsvn_repos`로 랩 된 버전을 사용할 필요가 있습니다 `svn_repos_fs_commit_txn`. 이 함수는 실제로는 만약 존재하면, 최초로 "pre-commit"폭스 클립트를 실행해, 그리고 트랜잭션(transaction)를 커밋해, 마지막에 "post-commit"폭스 클립트(을)를 실행합니다. 혹은, 실제로는 코어의 파일 시스템 프로그램 라이브러리 자신 에 포함되지 않는 특별한 보고의 구조를 준비합니다. (Subversion의 저장소(repository) 혹은 대한 자세한 것은>(을)를 봐 주세요).

혹의 구조는, 나머지의 파일 시스템 코드로부터 독립한 저장소(repository) 프로그램 라이브러리의 추상화가 하나의 이유입니다. `libsvn_repos` API 는 그 밖에도 몇개의 중요한 유틸리티를 Subversion에 제공하고 있습니다. 이것에는 이하와 같은 것이 있습니다:

1. Subversion 저장소(repository)와 거기에 포함되는 파일 시스템상에서의 파일의 생성, 오픈, 삭제, 그리고 회복의 스텝
2. 두 파일 시스템 트리간의 비교의 표시
3. 파일 시스템중에서 수정된 파일이 있는 모두(혹은 몇개의)의 리비전에 결합된 커밋 로그 메시지에의 문의
4. 파일 시스템의 가독인"덤프"의 생성, 파일 시스템중에 있다 리비전의 완전한 표현
5. 덤프 포맷의 해석, 다른 Subversion 저장소(repository)안에 덤프 된 리비전을 로드하는 것

Subversion가 계속 진화하는 것에 따라, 저장소(repository) 프로그램 라이브러리는 계속 증가하는 기능과 설정 가능한 옵션을 서포트를 제공한다 위해(때문에), 파일 시스템 프로그램 라이브러리와 함께 계속 과 함께 커진다 그렇지.

1.1.2. 저장소(repository) 액세스층

만약 Subversion 저장소(repository)층이,"통신로의 이제(벌써) 한편의 단 점" 이다면, 저장소(repository) 액세스층은, 그 통신로그 자체의 것입니다. 클라이언트 프로그램 라이브러리와 저장소(repository) 의 사이에 데이터를 상호 변환하는 것을 부과된 이 층은 `libsvn_ra`모듈 로더 프로그램 라이브러리, 그 RA모듈 자신(현재로서는, `libsvn_ra_dav`, `libsvn_ra_local`, 그리고 `libsvn_ra_svn`를 포함합니다), 그리고 하나 이상의 RA 모듈에 필요한 추가 의 프로그램 라이브러리, 예를 들어, `libsvn_ra_dav` 가 통신하기 위한 , `mod_dav_svn` Apache 모듈을 포함합니다. `mod_dav_svn` 모듈을 이용하지 않을 때에는, `libsvn_ra_svn` 의 서버인 **svnserve**가 통신합니다.

Subversion은, 저장소(repository) 리소스를 특정하는데 URL를 이용하므로, URL schema의 프로토콜부(보통은, `file:`, `http:`, `https:`, 혹은 `svn:`)는 어느 RA 모듈이 통신을 처리할까(을)를 결정하기 위해서(때문에) 사용됩니다. 각각의 모듈은, 프로토콜의 리스트(을)를 등록합니다만, 그것은 어떻게 이야기하면 좋은가를 알고 있으므로, RA 로더가 실행시에 어느 RA모듈을 그 처리를 위해서(때문에) 이용할까를 결정할 수가 있습니다. 어느 RA모듈이 Subversion 커멘드 라인 클라이언트에 이용 가능한가를 결정할 수가 있어 **svn --version**(을)를 실행하는 것으로, 어느 프로토콜은 서포트하고 있지 않다고 말해 올까(을)를 알 수가 있습니다. :

```
$ svn --version
```


svn, version 0.25. 0 (dev build)
compiled Jul 18 2003, 16:25:59

Copyright (C) 2000-2003 CollabNet.
Subversion is open source software, see <http://subversion.tigris.org/>

The following repository access (RA) modules are available:

- * ra_dav : Module for accessing a repository via WebDAV (DeltaV) protocol.
 - handles 'http' schema
- * ra_local : Module for accessing a repository on local disk.
 - handles 'file' schema
- * ra_svn : Module for accessing a repository using the svn network protocol.
 - handles 'svn' schema

1.1.2.1. RA-DAV (HTTP/DAV를 사용한 저장소(repository) 액세스)

libsvn_ra_dav 프로그램 라이브러리는, 서버와는 다른 머신상에서 실행되고 있는 클라이언트에 의해 이용되도록(듯이) 설계되고 있습니다. 클라이언트는 URL를 사용해 도달할 수가 있는 특정의 머신이다 그 서버와 통신합니다. 여기서 말하는 URL는, http: 또는 https: 의 프로토콜 부분을 포함하고 있는 것 같은 것입니다. 어떻게 이 모듈이 동작하는지를 이해하기 위해서, 최초로 저장소(repository) 액세스층의 특정의 설정중에 있는 것 외의 몇개의 키 컴퍼넌트에 접할 필요가 있습니다 그것은 강력한 Apache HTTP 서버와 Neon HTTP/WebDAV 클라이언트 프로그램 라이브러리입니다.

Subversion의 주된 네트워크 서버는 Apache HTTP 서버입니다. Apache 는 충분히 테스트되어 확장 가능한 open source의 서버 프로세스로, 그것은 성실한 용도에 이용할 수가 있습니다. 그것은 네트워크의 고부하에 유지할 수가 있어 많은 플랫폼상에서 동작합니다. Apache 서버는 많은 다른 표준 인증 프로토콜을 서포트해, 많은 사람들에게 의해 서포트된 모듈을 이용하는 것으로 확장할 수가 있습니다. 그것은 또 네트워크 파이프라인이나 캐싱 (와)과 같은 최적화를 서포트하고 있습니다. 서버로서 Apache 를 이용하는 것에 따라서, Subversion은 이러한 모든 기능을 자유롭게 손에 넣는 것이 할 수 있습니다. 그리고, (정도)만큼 어느 파이어 월(fire wall)는 HTTP의 통신을 통하도록(듯이) 설정되어 있으므로, 시스템 관리 책임자는, 보통은 파이어 월(fire wall) 설정 (을)를 변경할 필요조차 없게 Subversion을 동작시킬 수가 있습니다.

Subversion은 HTTP와 WebDAV(DeltaV 돌출하고)를 사용해, Apache 서버와 통신합니다. 이것에 대해서는, 이 장의 WebDAV의 마디를 불러 주세요. 그러나, 간단하게 말하면, WebDAV와 DeltaV 는 표준적인 HTTP1. 1 프로토콜 의 확장으로, 그것은 web상에서 파일의 공유와 버전화를 가능하게 합니다. Apache 2.0 은 mod_dav 가 준비되어 있어, 이것은 HTTP 의 DAV 확장을 이해하는 모듈입니다. Subversion 자신은 mod_dav_svn 를 서포트해 있습니다만, 이것은 다른 Apache 모듈로, mod_dav 와 협조해 동작해, (실제로는 그 연구 최종 단계로서) Subversion상에서의 구체적인 WebDAV와 DeltaV의 실장이 되고 있습니다.

HTTP 넘어로 저장소(repository)와 통신할 때, RA로더 프로그램 라이브러리는 libsvn_ra_dav 를 서버 프로세스 모듈로서 선택합니다. Subversion 클라이언트는 일반적인 RA 인터페이스를 호출해, libsvn_ra_dav 는 이러한 호출을(그것은 아직 대략적인 Subversion의 동작을 구체화합니다)을, HTTP/WebDAV 요구로 변환합니다. Neon 프로그램 라이브러리를 사용해, libsvn_ra_dav (은)는 이러한 요구를 Apache 서버에 송신합니다. Apache 는 이러한 요구 (을)를 받아(Web 브라우저가 하는 것과 완전히 같은 일반적인 HTTP 요구입니다만), DAV 관리의 위치로서 설정된 URL에 돌리고(httpd.conf파일중의 Location 인스트럭션을 사용합니다), 그 요구를 소유의 mod_dav 모듈에 건네줍니다. 적절히 설정되어 있으면, mod_dav (은)는 Apache 부속의 일반적인 mod_dav_fs 가 아니고, Subversion의 mod_dav_svn 를 파일 시스템에 관련한 요구에 대해서 이용하는 것을 알고 있습니다. 그래서, 마지막에는, 이 클라이언트는 mod_dav_svn 와 통신합니다만, 이것은 직접 Subversion 저장소(repository)층에 묶여 첨부 붙어 있다 물건입니다.

이것이 실제로 일어나는 교환의 간략화한 설명입니다. 예를 들어, Subversion 저장소(repository)는 Apache의 인증 인스트럭션에 의해 보호되고 있을지도 모릅니다. 이것에 의해, 저장소(repository)와 최초로 통신하려고 하는 시도가, 인증 붙어 있는 Apache에 따라서 실패에 끝날지도 모릅니다. 이 시점에서, libsvn_ra_dav는 아파치로부터, 불충분한 인증 밖에 얻을 수 없었기 때문에 클라이언트층에 갱신된 인증 데이터를 취득하기 위해서 콜백 했다, 라고 하는 통지를 받습니다. 만약 이 데이터가 올바르게

취득할 수 있으면, 유저는, 허가된 최초의 조작 (을)를 실행하는, libsvn_ra_dav의 다음의 아토믹인 요구를 찾아, 모두 하지만 잘됩니다. 만약 충분한 인증 정보가 주어지지 않으면 요구는 최종적으로 실패해, 클라이언트는 유저에게 그 취지를 보고합니다.

Neon와 Apache를 사용해, Subversion은 다른 여러가지 복잡한 area에의 자유로운 기능을 얻을 수도 있습니다. 예를 들어, 만약 Neon가 OpenSSL 프로그램 라이브러리 (을)를 찾아냈을 경우, 그것은 Subversion 클라이언트에 SSL로 암호화되었다 통신을, Apache 서버로 하는 것을 인정합니다. (그 고유의 mod_ssl 는"그 언어를 이야기합니다"). 또, Neon 자신과 Apache의 mod_deflate는"deflate"알고리즘을 이해할 수 있는 프로(PKZIP와 gzip 프로그램으로 이용되고 있는 것과 같은 것입니다만), 요구는 보다 작은 압축된 덩어리로서 통신로를 흐릅니다. Subversion가 향후 서포트하고 싶다고 생각하고 있는 것 외의 복잡한 기능 (으)로서는, 자동적으로 서버측의 리디렉트를 처리 하는 것(예를 들어, 저장소(repository)가 다른 새로운 URL로 이동한 것 같은 경우)나, HTTP 파이프라인 의 혜택을 받는 것, 등입니다.

1.1.2.2. RA-SVN (고유의 프로토콜에 의한 저장소(repository) 액세스)

표준적인 HTTP/WebDAV 프로토콜에 가세해, Subversion은 고유의 프로토콜을 사용하는 RA의 실장도 준비해 있습니다. libsvn_ra_svn 모듈은 고유의 네트워크 소켓 접속을 실장해, 저장소(repository)가 있는 스탠드얼론 서버 (와)과 통신하는 svnserve입니다 클라이언트는svn:// schema로 저장소(repository)에 액세스로 옵니다.

이 RA 실장은, 전의 마디로 접한 Apache의 이점의 대부분이 부족하고 있습니다. 그러나, 그것은 어떤 종류의 시스템 관리 책임자를 끌어당길지도 모릅니다. 그것은 매우 간단하게 설정해 실행할 수 있습니다. svnserve 프로세스의 설정은, 거의 순간적으로 끝 납니다. 또 그것은 Apache보다 (코드행수라고 하는 의미로) 훨씬 작고, 시큐리티나 다른 사정에 의한다 체크도 훨씬 용이합니다. 한층 더 몇개의 시스템 관리 책임자는 벌써 SSH 의 시큐리티 인프라를 가지고 있어, Subversion에도 그것을 사용하게 하고 싶다 (이)라고 생각하고 있을지도 모릅니다. ra_svn 를 사용하는 클라이언트는 SSH 를 개입시켜 프로토콜을 간단하게 터널 할 수가 있습니다.

1.1.2.3. RA-Local (저장소(repository)에의 직접적인 액세스)

Subversion 저장소(repository)와의 모든 통신이 큰 서버 프로세스와 네트워크층 (을)를 필요로 하는 것은 아닙니다. 로컬 디스크 우에노 저장소(repository)에 액세스 하고 싶은 것뿐의 유저에게 있어서는,file: 를 사용할 수가 있어 libsvn_ra_local가 제공하는 기능을 사용할 수가 있습니다. 이 RA 모듈은 직접 저장소(repository)와 파일 시스템 프로그램 라이브러리와 결합되므로, 네트워크 통신은 전혀 필요 없습니다.

Subversion은file: URL의 일부로서 localhost 인가, 하늘을 인 서버 명칭을 포함하는 것을 요구해, 포트 지정은 없습니다. 말투를 바꾸면(자), URL는 무엇인가, file:///localhost/path/to/repos 인가 file:///path/to/repos (와)과 같은 형태의 것이 됩니다.

게다가 Subversion의file:URL는 보통 web 브라우저가 file:URL가 하는 방법에서는 이용할 수 없는 것에 주의해 주세요. 통상의 web 브라우저로 file:URL를 열람하려고 말할 때, 파일 시스템을 직접 조사하는 것으로 그 자리소에 있는 파일의 내용을 읽어 들여 표시합니다. 그러나, Subversion의 리소스는 가상 파일 시스템중에 있어, (>참조) 당신의 브라우저는 그 파일 시스템을 어떻게 읽으면 좋은가 이해할 수 없을 것입니다.

1.1.2.4. Your RA Library Here

한층 더 다른 프로토콜을 사용해 Subversion의 저장소(repository)에 액세스 하고 싶으면 말하는 사람에게 있어야만, 어째서 저장소(repository) 액세스층이 모듈화되고 있다 일까하고 말하는 이유가 됩니다. 개발자는 다른 한쪽으로 RA 인터페이스를 실장한다 새로운 프로그램 라이브러리를 간단하게 쓸 수가 있어 이제(별써) 한편으로 그 저장소(repository)와 통신할 수가 있습니다. 새로운 프로그램 라이브러리는 이미 존재하고 있는 네트워크 프로토콜을 이용할 수도 있고, 스스로 개발한 것이라도 좋습니다. 프로세스간 통신(IPC) 호출을 사용할지도 모르지않고조금 바보 일지도 알려지지 않습니다 만e-mail 베이스의 프로토콜을 사용하는 것도 할 수 있습니다. Subversion은 API를 제공해, 당신은 자신의 상상성을 제공한다, 라고.

1.1.3. 클라이언트측

클라이언트측에서 보면(자), Subversion의 작업 카피는 모든 처리가 일어나는 장소입니다. 클라이언트측 프로그램 라이브러리에 의해 실장되는 기능은, 작업 카피의 관리라고 하는 다만 하나의 목적으로 위해(때문에) 존재합니다로컬인 장소에 하등의 형태로 제공된다 파일과 다른 서브 디렉토리가 있는 디렉토리가, 하나 이상의 저장소(repository) 위치를 반영한 것으로 해, 저장소(repository) 액세스층과의 사이의 변경을 전하거나 합니다.

Subversion의 작업 카피 프로그램 라이브러리, libsvn_wc 는 작업 카피중의 데이터의 관리에 직접적인 책임을 집니다. 이것을 하기 위해서(때문에), 이 프로그램 라이브러리(은)는 특별한 서브 디렉토리아에 각각의 작업 카피에 대한 관리 정보 (을)를 격납합니다. 이 서브 디렉토리는, svn 라고 한다 이름입니다만, 어느 작업 카피중에도 존재해, 관리에 관계한 동작을 하기 위한 상태를 기록해, 작업 스페이스를 확보하기 위한 다양한 파일이나 디렉토리를 포함하고 있습니다. CVS에 친숙함이 있는 사람이라면, 이, svn 서브 디렉토리는, 그 목적으로 해 CVS의 작업 카피에 있는 관리 디렉토리CVS 에 잘 비슷한 것을 알 수 있다고 생각합니다. . svn 관리 area에 대한 자세한 것은, 이 장의 > (을)를 참조해 주세요

Subversion 클라이언트 프로그램 라이브러리 libsvn_client 는 광범위의 역할을 집니다. 그 일은, 작업 카피 프로그램 라이브러리의 기능과 저장소(repository) 액세스층 의 기능을 묶는 것으로, 일반적인 리비전 제어를 실행하고 싶다고 생각한 모든 어플리케이션에 최상정도의 API를 제공하는 것입니다. 예를 들어 svn_client_checkout 함수는 인수로서 URL를 취합니다. 이 함수는 URL를 RA층에 나, 특정의 저장소(repository)에 인증되었다 세션을 엽니다. 그리고 그 저장소(repository)에 특정의 트리를 지정해, 이 트리를 작업 카피 라이브러리에 보냅니다만, 이번은 그 프로그램 라이브러리가 작업 카피 전체를 디스크에 기입합니다(. svn 디렉토리를 포함한 모든 정보).

클라이언트 프로그램 라이브러리는 어떠한 어플리케이션으로부터도 이용할 수 있다 같게 설계되고 있습니다. Subversion의 원시 코드는 표준적인 커멘드 라인 클라이언트를 포함하고 있으므로, 그 클라이언트 프로그램 라이브러리의 위에 좋아할 뿐(만큼) GUI 클라이언트를 쓸 수가 있습니다. Subversion의 새로운 GUI(혹은 실제로는 새로운 클라이언트)는, 커멘드 라인 클라이언트를 포함한 촌티 있고 래퍼일 필요는 없습니다. 그 것은, libsvn_client API 를 통해서 커멘드 라인 클라이언트 하지만 사용하고 있는 것과 같은 기능, 데이터, 콜백의 구조에 완전하게 액세스 할 수가 있습니다.

직접적인 바인드정확함에 대한 말

왜 GUI 프로그램은, 직접 libsvn_client 에 바인드 해, 커멘드 라인 프로그램을 감싸는 래퍼로서 동작하지 않는 것일까요? 그것은 단지보다 효율적이기 때문이다라는 것만으로는 없고, 잠재적인 정확함에 대한 문제도 있습니다. 커멘드 라인 프로그램(Subversion가 제공하고 있는 것 같은 물건)은 클라이언트 프로그램 라이브러리에 이어 붙어 있습니다만, C언어의 형태를 가지는 피드백과 데이터 비트의 요구를, 인간이 읽을 수 있는 형태의 출력에 효율적으로 변환할 필요가 있습니다. 이 손의 변환은 부정확하게 되기 쉽습니다. 즉, 프로그램은 API (으)로부터 취득한 정보의 모든 것을 표시하지 않을지도 모르고, 요약한 표현 형식 (이)가 되도록(듯이) 정보를 이어 대면시키거나 할지도 모릅니다.

그러한 커멘드 라인 프로그램을 다른 프로그램으로 랩 하면(자), 랩 하는 편의 프로그램은 벌써 회사 되었다(그리고 주의한 것처럼 아마 불완전한) 정보에 액세스 하는 것이 가능한 한으로, 그것은 **한번 더, 자신에게 소유의** 표현 형식으로 변환하지 않으면 안됩니다. 각각의 랩핑의 층 마다, 최초의 데이터의 정확함은 자꾸자꾸 없어져 갈 가능성이 있어, 그것은 정확히 자신이 좋아하는 오디오나 비디오카세트를 반복해 카피할 경우에 일어나는 것 같은 이야기가 되어 버립니다.

1.2. API의 이용

Subversion 프로그램 라이브러리 API를 사용한 어플리케이션의 개발은 비교적 순진한 형태로 진행됩니다. 모든 헤더 파일은 소스 트리의 subversion/include 에 있습니다. 이러한 헤더는 원시 코드로부터 Subversion을 만들어 인스톨 하면(자), 그 머신의 시스템 헤더의 두는 곳소에 카피됩니다. 이러한 헤더에는 Subversion 프로그램 라이브러리의 유저에 의해 액세스 할 수 있는 것 같은 기능과 형태의 모두가 있습니다.

최초로 조심하지 않으면 안 되는 것은 Subversion의 데이터형과 함수는 고유의 이름 공간에 의해 분리되어 있는 것입니다. 모든 공공적인 Subversion 심볼명은 "svn_"로 시작되어, 그 심볼이 정의되고 있는 프로그램 라이브러리의 짧은 코드가 계속되어, ("wc"라든지, "client"라든지, "fs"등), 언더 스코아가 하나 와, ("_"), 마지막에 심볼명의 나머지의 부분이 옵니다. 한정적으로 공공적인 함수(프로그램 라이브러리중의 원시 파일 사이에서는 이용되지만, 프로그램 라이브러리의 밖에서는 이용되지 않고, 프로그램 라이브러리 디렉토리 자신의 내부에서만 참조 가능한 것)은 이 명명 규약과는 차이, 프로그램 라이브러리 코드 의 후에 언더 스코아가 하나 오는 대신에, 둘 옵니다 ("__"). 어느 원시 파일로 사적인 함수는 특수한 접두사를 가지지 않고, static선언됩니다. 물론 컴파일러는 이러한 명명 규약을 해석합니다만, 어느 함수의 스코프나 데이터형을 분명히 하는데 큰 도움이 됩니다.

1.2.1. Apache Portable Runtime 프로그램 라이브러리

Subversion 자신의 데이터형과 함께, "apr_"로 시작되는 데이터형에의 참조를 많이 보이는 일이 있는 이것은 Apache 의 Portable Runtime (APR) 프로그램 라이브러리입니다. APR 는 Apache 의 가반인 프로그램 라이브러리입니다만, 원래 Apache의 서버 코드의 OS의존의 부분을 OS비의존의 부분으로부터 분리하기 위해서 만들어졌습니다. 결과, OS 마다, 조금, 혹은 크게 다른 조작을 실행한다 유익의 추상적인 API를 제공하게 되었습니다. Apache HTTP 서버는 분명하게 APR 프로그램 라이브러리의 최초의 이용자였지만, Subversion 개발자는 곧바로 APR를 사용하는 것의 중요성을 눈치챈습니다. 이것은 실제로 Subversion 자신중에 전혀 OS에 의존하고 있지 않는 코드의 부분이 있는 것을 의미 합니다. 게다가 Subversion 클라이언트는 서버가 컴파일 해 실행하는 장소이면 어디에서라도 실행할 수 있는 것을 의미합니다. 현시점에서는 이러한 OS 에는, Unix 좋아하는 모두, Win32, BeOS, OS/2 그리고 Mac OS X 하지만 포함됩니다.

operating system간에 다른 시스템 콜의 일관했다 실장을 제공하는 것에 입에 물어, [3] APR 는 Subversion가 많은 독자적인 데이터형에 직접 액세스 하는 것을 가능하게 합니다만, 거기에는, 동적인 배열이나 해시 테이블이 있습니다. Subversion 는 원시 코드중에서 이러한 형태를 확장해 이용합니다. 그러나, 아마 가장 광범위하게 이용되고 있는 APR 데이터형은, 대부분 모든 Subversion API prototype에 나타납니다만, apr_pool_t입니다 APR의 메모리프르입니다. Subversion 는 풀을 내부적으로 모든 메모리 확보 하지만 필요한 경우에 이용합니다. (다만, 외부 프로그램 라이브러리가 그 API를 통해서 주고 받는 데이터의 메모리 매니지먼트에 이것과 다른 형식을 요구하지 않는 한에 두어, 입니다.) [4] 그리고, Subversion API 에 대한 코딩은 같은 것이 요구된다 (뜻)이유가 아닙니다만, 필요한 장소에서는 API 함수를 위해서(때문에) pool 를 준비한다 (일)것은 요구됩니다. 이것은 APR 도 링크 할 필요가 있는 Subversion API 의 유저는 apr_initialize()를 불러 APR 하부조직 (을)를 초기화할 필요가 있어, 그리고 Subversion API 호출을 이용하기 위해서 pool 를 준비하지 않으면 안 되는, 이라는 것이 됩니다. 자세한 것은 >(을)를 봐 주세요.

1.2.2. URL 와 Path 의 요구

Subversion 전체의 문제로서의 리모트 버전 관리 조작에서는, 국제화(i18n) 의 서포트를 따라가는등인가 주의해 둘 필요가 있습니다. 결국, "리모트"가, "오피스 이외의 장소"를 의미한다면, 그것은 "전세계로부터"라고 하는 의미이기도 합니다. 이러한 상황에 대응하기 위해서 Subversion의 패스 인수를 취하는, 모든 퍼블릭 인터페이스는 패스가 정규화되어 UTF-8으로 encode 되고 있는 것으로 합니다. 이것은 예를 들어, libsvn_client 인터페이스를 호출하는, 새로운 클라이언트 바이너리는 모두, Subversion 프로그램 라이브러리에 패스를 건네준다 전에, 우선 패스를 로컬 코딩으로부터 UTF-8으로 변환하지 않으면 안되어, Subversion 로부터의 결과 패스를, Subversion 이외의 목적으로 이용하기 전에는 로컬 코딩에 재변환하지 않으면 안 된다고 하는 것입니다. 행운의 일로, Subversion은 이러한 변환이 필요한 임의의 프로그램이 이용할 수 있는 것 같은 함수를 준비해 있습니다 (subversion/include/svn_utf.h를 참조해 주세요).

또, Subversion API 는 모든 URL 인수가 올바르게 URI encode 되고 있다 것을 요구합니다. 그래서 My File.txt라는 이름의 파일 URL를, file:///home/username/My File.txt (와)과 건네주는 대신에, file:///home/username/My%20File.txt (와)과 건네주지 않으면 안됩니다. 역시 Subversion은 어플리케이션을 이용할 수 있는 헬퍼 함수를 준비해 있습니다 svn_path_uri_encode 와 svn_path_uri_decode를 사용해 각각 URI 의 encode와 디코드를 할 수 있습니다.

1.2.3. C 와 C++ 이외의 언어의 이용

C언어 이외의 것과 조합해 Subversion 프로그램 라이브러리를 사용하는데 흥미가 있다면 예를 들어 Python의 스크립트나 Java 어플리케이션 Subversion은 Simplified Wrapper and Interface Generator (SWIG)라고 하는 형태 그리고 조금 서포트하고 있습니다. Subversion용의 SWIG는, subversion/bindings/swig에 있어, 천천히 이용 가능한 상태로 자라고 있습니다. 이것을 사용하면, 스크립트 언어 고유의 데이터형을, Subversion의 C프로그램 라이브러리로 필요한 데이터형으로 변환하는 나팔을 사용해 Subversion API 를 간접적으로 호출할 수가 있게 됩니다.

언어 제휴를 통해서 SubversionAPI 에 액세스 하는 것은 분명하게 이점이 있습니다 단 순함, 쉽다. 일반적으로, Python 나 Perl 라고 하는 언어는 C 나 C++ (을)를 사용하는 것보다도 훨씬 유연하고 쉬운 것입니다. 이러한 언어가 준비해 있다 고레벨 데이터 형과 문맥 의존의 데이터형의 체크와 같은 것은, 좀 더 잘 유저로부터의 정보를 처리합니다. 벌써 아시는 바와 같이, 인간만이 프로그램의 입력을 잘못해 스크립트계의 언어는 단지 그 사이 차이를 좀 더 잘 처리할 뿐입니다. 물론, 자주 이 유연성은 퍼포먼스를 희생에 섬. 이것이, 왜 고도로 최적화된 C베이스의 인터페이스와 프로그램 라이브러리를, 강력하고 유연한 언어와 조합해 이용하는 것에 사람이 끌어 당기고 인가의 이유입니다.

Subversion의 Python용 SWIG 제휴의 예를 봅시다. 이 예는 전의 예와 같은 것을 합니다. 이번 함수의 사이즈와 복잡함의 정도에 주의합니다.

Example 1-2. Python를 사용한 저장소(repository)층

```
from svn import fs
import os.path

def crawl_filesystem_dir (root, directory, pool):
    """Recursively crawl DIRECTORY under ROOT in the filesystem, and return
    a list of all the paths at or below DIRECTORY. Use POOL for all
    allocations. """

    # Get the directory entries for DIRECTORY.
    entries = fs.dir_entries(root, directory, pool)

    # Initialize our returned list with the directory path itself.
    paths = [directory]

    # Loop over the entries
    names = entries.keys()
    for name in names:
        # Calculate the entry's full path.
        full_path = os.path.join(basepath, name)

        # If the entry is a directory, recurse. The recursion will return
        # a list with the entry and all its children, which we will add to
        # our running list of paths.
        if fs.is_dir(fsroot, full_path, pool):
            subpaths = crawl_filesystem_dir(root, full_path, pool)
            paths.extend(subpaths)

        # Else, it is a file, so add the entry's full path to the FILES list.
        else:
            paths.append(full_path)

    return paths
```

전의 예에서의 C 에 의한 실장은 좀 더 훨씬 긴 것이었습니다. C 의 같은 routine (은) 는 메모리의 이용의 방법에 신경을 사용할 필요가 있어, 엔트리의 해시와 패스의 리스트를 표현하는 독자적인 데이터형이 필요했습니다. Python 는 해시와 리스트(각각 "dictionaries"와 "sequences"라고 하는 말투를 합시다만)를 편입 데이터형이라고 해두는 있어, 이러한 형태에 대한 훌륭한 조작 방법 하지만 준비되어 있습니다. 그리고, Python 는 참조 카운트와 쓰레기 콜렉션 (을)를 사용하므로, 이 언어의 유저는 메모리의 확보와 개방에 대해 고민할 필요가 없습니다.

이 장의 전의 마디로, `libsvn_client` 인터페이스에 접해 Subversion 클라이언트를 쓰기 위한 노력을 경감한다고 하는 유일한 목적을 위해서(때문에) 있다고 했습니다. 이하는 이 프로그램 라이브러리에, 어떻게 해 SWIG 제휴를 이라고 눌러 액세스 할 수가 있을까의 간단한 예입니다. Python의 몇 줄기의 코드로, 완전하게 기능하는 Subversion의 작업 카피를 체크 아웃 할 수가 있습니다.

Example 1-3. 작업 카피를 체크아웃 하는 단순한 스크립트

```
#!/usr/bin/env python
import sys
from svn import util, _util, _client

def usage():
    print "Usage: " + sys.argv[0] + " URL PATHWn"
    sys.exit(0)

def run(url, path):
    # Initialize APR and get a POOL.
    _util.apr_initialize()
    pool = util.svn_pool_create(None)

    # Checkout the HEAD of URL into PATH (silently)
    _client.svn_client_checkout(None, None, url, path, -1, 1, None, pool)

    # Cleanup our POOL, and shut down APR.
    util.svn_pool_destroy(pool)
    _util.apr_terminate()

if __name__ == '__main__':
    if len(sys.argv) != 3:
        usage()
    run(sys.argv[1], sys.argv[2])
```

현시점에서, 이것이 Subversion의 Python 제휴이며, 그것은 거의 완성 된 것입니다. Java 제휴에 대해서도 조금 접해 둡니다. SWIG 인터페이스 파일이 올바르게 설정되면, 모든 SWIG 대응 언어(현재로서는, Tcl, Python, Perl, Java, Ruby, Mzscheme, Guile, 그리고 PHP입니다만)의 특정의 나팔을 생성하는 것은 이론적으로는 간단한 일입니다. 그러나, SWIG가 일반화하는 도움이 필요한 복잡한 API에 대해서는, 좀 더 추가의 코딩 하지만 필요합니다. SWIG 자신의 것보다 자세한 정보는 <http://www.swig.org>에 있는 프로젝트의 웹 사이트를 봐 주세요.

1.3. 작업 카피 관리 area의 내부

이전 지적인 것처럼, Subversion 작업 카피의 디렉토리의 각각은 `.svn` 라는 이름의 특별한 서브 디렉토리를 갖고, 거기에 작업 카피 디렉토리에 관한 관리 정보를 격납합니다. Subversion는 `.svn` 중의 정보를 이하와 같은 일을 기록하는데 이용합니다:

- 어디에 있는 저장소(repository)가, 작업 카피 디렉토리의 파일이나 서브 디렉토리에 의해 표현되고 있는 것인가.
- 어느 리비전의 파일이나 디렉토리가 현재의 작업 카피에 있는 것인가.
- 파일이나 디렉토리에 이어 붙은 유저 정의의 속성.
- 작업 카피 파일의 수정원(편집전) 카피. files.

`.svn` 디렉토리에 격납된 데이터에는 그 밖에도 여러 가지 있습니다만, 가장 중요한 아이템의 몇개인가인 만큼 대해 설명합니다.

1.3.1. Entries 파일

`.svn` 디렉토리에 있는 제일 중요한 파일은 entries 파일입니다. 이 파일은 XML 문서

로 그 내용은 작업 카피 디렉토리의 버전 관리하에 있는 리소스에 대한 관리 정보의 모음입니다. 저장소(repository) URL, 수정원리비전, 파일의 체크 섬, 수정원 텍스트와 속성의 타임 스탬프, 예고와 충돌 상태에 관한 정보, 마지막에 커밋했던 것에 관한 정보(실행자, 리비전, 타임 스탬프), 로컬 카피 히스토리Subversion 클라이언트가 관리하고 있는 리소스 에 붙어 흥미가 있는 정보는 모두 여기에 기록되고 있습니다.

Subversion와 CVS의 관리 area의 비교

전형적인. svn디렉토리의 내부를 보면(자), 그것은CVS 의 관리 디렉토리에서 CVS가 관리 하는 정보보다, 조금 많은 것을 압니다. entries 파일은 현재의 작업 카피 디렉토리 상태를 기술한 XML를 포함하고 있어, 이것은 기본적으로 CVS의 Entries와 Repository (을)를 함께 한 것이 됩니다.

이하는, 실제의 entries 파일의 예입니다:

Example 1-4. 전형적인. svn/entries 파일

```
? xml version="1.0" encoding="utf-8"?
wc-entries
  xmlns="svn:"
entry
  committed-rev="1"
  name="svn:this_dir"
  committed-date="2002-09-24T17:12:44. 064475Z"
  url="http://svn.red-bean.com/tests/.greek-repo/A/D"
  kind="dir"
  revision="1"/
entry
  committed-rev="1"
  name="gamma"
  text-time="2002-09-26T21:09:02. 000000Z"
  committed-date="2002-09-24T17:12:44. 064475Z"
  checksum="QSE4vWd9ZM0cMvr7/+YkXQ=="
  kind="file"
  prop-time="2002-09-26T21:09:02. 000000Z"/
entry
  name="zeta"
  kind="file"
  schedule="add"
  revision="0"/
entry
  url="http://svn.red-bean.com/tests/.greek-repo/A/B/delta"
  name="delta"
  kind="file"
  schedule="add"
  revision="0"/
entry
  name="G"
  kind="dir"/
entry
  name="H"
  kind="dir"
  schedule="delete"/
/wc-entries
```

알도록(듯이), entries 파일은 본질적으로는 엔트리의 리스트입니다. entry 태그의 각각은 3개 중 어떤 것인지를 표현하고 있는: 작업 카피 디렉토리 자신(name 속성을 "svn:this-dir"로 설정하는 것으로 가리킵니다), 작업 카피 디렉토리에 있는 파일(kind 속성을 "file"로 설정하는 것으로 가리킵니다), 혹은 작업 카피의 서브 디렉토리(kind 가 여기에서는 "dir" (으)로 설정됩니다). 엔트리가 이 파일에 격납되는 파일과 서브 디렉토리는 벌써 버전 관리하에 있을까(위의 예의 zeta 파일과 같이), 이 작업 카피 디렉토리의 변경이 다음에 커밋될 때 버전 관리하에 추가하는 것이 예고되고 있는

지,입니다. 엔트리의 각각은 독특한 이름을 갖고, 특정의 노드 종별을 가집니다.

개발자는, Subversion 가entries 파일을 읽고 쓰기할 경우에 사용하는 특별한 규칙에 주의해야 합니다. 모든 엔트리는 자신의 리비전과 이어 붙어 있는 URL를 가지고 있습니다만, 샘플 파일중의 모든entry 태그가 명시적인revision 나 url 속성을 가지는 것은 아닙니다. Subversion 는 엔트리가 명시적으로 이 두 개의 속성을 가지지 않는 것도 인정하고 있습니다만, 그것은, 그 값이, "svn:this-dir" 엔트리에 있는 데이터와 같은가, 간단하게 계산할 수 있는 경우입니다. [5] 또, 서버 디렉토리의 엔트리에 대해서는, Subversion 는 중요한 속성이름, 종별, url, 리비전, 그리고 예고 상황 만을 보존하기로 주의해 주세요. 중복 하는 정보를 줄이기 위해서(때문에), Subversion 는, 서버 디렉토리에 관한 완전한 정보 (을)를 결정하는 방법으로서 그 서버 디렉토리에 나와 가 그 디렉토리 자신의. svn/entries 파일의 "svn:this-dir" 엔트리를 읽도록(듯이) 지시합니다. 그러나, 서버 디렉토리에의 참조는, 그 부모의 entries 파일에 기록되고 있어, 서버 디렉토리 하지만 디스크로부터 삭제되어 버린 것 같은 경우에서도 기본적인 버전 관리 조작을 하는데는 충분한 정보를 가지고 있습니다.

1.3.2. 수정원카피와 속성 파일

앞으로 주의한 것처럼,. svn디렉토리는 또 수정원의"text-base"버전의 파일을 보존하고 있습니다. 이것은. svn/text-base에 있습니다. 수정원 카피의 이점은, 몇개인가 있습니다. 네트워크의 통신없이 로컬 수정을 체크해 차분을 보고하거나 네트워크 통신없이 수정하거나 삭제한 파일을 바탕으로 되돌리거나 서버에의 변경점의 송신 사이즈를 줄이거나 할 수 있습니다그러나, 적어도 각각의 버전 관리된 파일을 둘디스크상에 보존하는 코스트가 발생합니다. 최근에는, 이것은(정도)만큼 어느 파일에 대해 무시할 수 있는 정도의 물건입니다. 그러나, 버전 관리된 파일이 커지는 것에 따라는 이 상황은 심한 것에 되어 갑니다. "text-base" 의를 만드는지 어떤지를 옵션으로서는, 이라고 하는 의견도 있습니다. 그러나 짓궂게도, 버전 관리하는 파일의 사이즈가 커지는에 따라,"텍스트 베이스"의 존재도, 그 만큼 중요하게 되어 가는 파일로 한 아주 조금의 변경을 커밋하고 싶은 것뿐인데, 엄청나게 크다 파일을 네트워크 넘어로 보내자는, 누가 생각할까요?

"text-base"파일과 같은 목적으로, 속성 파일과 그 수정원 "prop-base"카피가 있습니다. 각각,. svn/props (와)과. svn/prop-base 에 있습니다. 디렉토리도 속성을 가질 수가 있으므로,. svn/dir-props 와 . svn/dir-prop-base 파일이 있습니다. 이러한 속성 파일의 각각("작업중"과"원래의"버전)은 속성명과 속성치를 격납하는데, 단순한"디스크상 해시"파일 형식 (을)를 사용합니다.

1.4. WebDAV

WebDAV ("Web 베이스의 분산 편집과 버전화")는 표준적인 HTTP 프로토콜의 확장으로, 기본적으로는 읽어들이 전용의 매체인 web를, 읽고 쓰기 가능한 매체와 하기 위해서 설계되었습니다. 생각으로서는, 디렉토리와 파일은 읽고 쓰기 가능한 오브젝트로서Web상에서 공유할 수 있다고 한다 물건입니다. RFCs 2518 으로 3253 은, HTTP의 WebDAV/DeltaV 확장에 대해 기술 되고 있어, (다른 유용한 정보와 함께) <http://www.webdav.org/>로 입수 가능합니다.

몇개의 operating system의 파일 브라우저는 벌써 WebDAV를 사용한 네트워크 디렉토리를 mount 할 수가 있습니다. Win32에서는, Windows Explorer 는 "Web 폴더"(그것은 확실히, WebDAV 하지만 준비한 네트워크의 장소입니다만)이라고 부르고 있는 것을, 마치 그것이 보통 공유 폴더인것 같이 참조할 수가 있습니다. Mac OS X 도 이 능력이 있어, Nautilus 나 Konqueror 브라우저도 그렇습니다. (이것들은, GNOME 와 KDE 상에서 각각 움직입니다).

이것들 모든 것은 어떻게 해 Subversion 에 적용되고 있는 것일까요? mod_dav_svn Apache 모듈은 그 주요한 네트워크 프로토콜로서 WebDAV와 DeltaV로 확장된 HTTP를 사용하고 있습니다. 새로운 특별한 프로토콜을 실장하는 것보다도, 초기의 Subversion의 설계자는, 단지 Subversion로 사용한다 버전과 처리의 개념을 RFC 2518 으로 3253 이 요구하는 개념에 맞추었습니다. [6]

WebDAV의 것 좀 더 철저한 논의, 어떻게 동작해, Subversion은 그것을 어떻게 사용하는지, 에 대해서는,>을 봐 주세요. 다른 화제와 함께, Subversion 가 어느 정도 일반적인 WebDAV 의 사양을 계승하고 있어 일반적인 WebDAV 클라이언트 (와)과의 상호 운용성에 어떤 영향을 줄까에 대한 논의가 있습니다.

1.5. 메모리프르를 사용한 프로그래밍

C 언어를 사용한 것이 있는 거의 모든 개발자는, 어떤 시점에 메모리 관리로 진절머리 나고 있었던 쟈 한숨 돌리는 일이 있겠지요. 이용하기 위해서 필요한 충분한 메모리를 확보해, 그 이용 상황을 기록해, 필요없게 되면(자) 메모리를 해방하는그러한 처리는 매우 복잡합니다. 그리고 물론, 거기에 실패하면(자), 프로그램이 망가져 끝내, 심할 때에는 컴퓨터가 고장나 버립니다. 행운에도, Subversion가 가반성을 위해서(때문에) 이용하고 있는 APR 프로그램 라이브러리 (은)는apr_pool_t형을 준비해 있어, 이것은 메모리의"풀"을 표현하는 것입니다.

메모리프르는 프로그램에 의해 이용하기 위해서 확보된 메모리 블록의 추상적인 표현입니다. 표준적인malloc() 함수 (와)과 그 아중을 사용해 OS로부터 직접 메모리를 취득하는 대신에, APR를 링크 했다 프로그램은 단지 메모리 풀을 만드는 요구를 내는 것으로 실시합니다. (이것에는apr_pool_create() 함수를 사용합니다) APR는 OS로부터 자연스러운 사이즈의 메모리를 확보해, 그 메모리는 곧바로 프로그램 그리고 사용할 수가 있게 됩니다. 프로그램이 풀 메모리를 필요와 할 경우에는 언제라도,apr_palloc()와 같은 APR 풀 API 함수의 어떤 것인지를 사용할 수가 있어, 그것은 풀로부터 범용적인 메모리를 확보해 돌려줍니다. 프로그램은 요구 비트와 풀로부터의 메모리를 계속 요구할 수가 있어, APR는 그 요구를 계속 승인할 수가 있습니다. 풀은 프로그램에 아울러 자동적으로 사이즈가 많은구든지, 최초 풀에 포함되어 있었던 것보다도 많은 메모리를 요구할 수가 있습니다. 이것은 시스템에 메모리가 없어질 때까지 계속할 수가 있습니다.

이것으로 풀의 이야기가 마지막이라면, 특별한 주위를 기울일 필요도 없습니다만. 행운에도, 그렇지는 않습니다. 풀은 만들어지는 것 만으로는 아닙니다: 그것은 또 클리어 하거나 삭제할 수도 있습니다. 이것에는 apr_pool_clear() 와 apr_pool_destroy() 를 각각 이용합니다. 이것은 개발자에게 얼마든지의혹은 몇천의area를 풀로부터 취득해, 그 후 한 번의 함수 호출해, 그 모두 클리어 하는 유연성을 줍니다. 게다가 풀은 계층을 가지고 있습니다. 벌써 만들어진 어느 풀 에도"서브 풀"을 만들 수가 있습니다. 풀이 클리어 되면(자), 그 풀 의 모든 서브 풀은 삭제됩니다. 만약 풀을 삭제하면(자), 그 풀 자신과 서브 풀의 양쪽 모두가 삭제됩니다.

먼저 진행하기 전에, 개발자는 Subversion 원시 코드중에, 지금 말한 것 같은 APR 풀 함수의 호출이, 그만큼 많지 않은 것에 눈치채겠지요. APR 풀은, 몇개의 확장 메카니즘을 갖고 있어, 그것은 풀에 국유의 유저 데이터를 접속하는 능력이나, 풀이 삭제될 때 불러 가는 클린 업 함수를 등록하는 구조등이 있습니다. Subversion은 이러한 확장 기능을, 그만큼 자명하지 않는 방법으로 이용합니다. 그래서 Subversion 는(그리고 그 코드를 사용하는 사람의 대부분은) 나팔 함수 이다 svn_pool_create(), svn_pool_clear(), 그리고 svn_pool_destroy() (을)를 제공하고 있습니다.

풀은 기본적인 메모리 메니지먼트에도 도움이 됩니다만, 루프나 재귀적인 상황으로 의 풀의 구축은 정말로 훌륭한 것입니다. 루프는 자주 그 반복 회수가 부정이며, 재귀적은 그 깊이가 부정 그래서, 이러한 area에서의 코드의 메모리 소비량은 예측일궤 선. 행운에도, 네스트 한 메모리프르를 사용하면(자), 이러한 잠재적인 무서운 상황을 간단하게 관리할 수가 있습니다. 이하의 예는, 자주 있는 매우 복잡한 정보에서의 네스트 한 풀의 기본적인 사용법을 나타내고 있습니다. 이 상황이란, 디렉토리 트리를 재귀적으로 더듬으면서, 트리의 모든 장소인 처리를 실행한다, 라고 한 것입니다.

Example 1-5. 효율적인 풀의 이용

```
/* Recursively crawl over DIRECTORY, adding the paths of all its file
   children to the FILES array, and doing some task to each path
   encountered. Use POOL for the all temporary allocations, and store
   the hash paths in the same pool as the hash itself is allocated in. */
static apr_status_t
crawl_dir (apr_array_header_t *files,
           const char *directory,
           apr_pool_t *pool)
{
    apr_pool_t *hash_pool = files->pool; /* array pool */
    apr_pool_t *subpool = svn_pool_create (pool); /* iteration pool */
    apr_dir_t *dir;
    apr_finfo_t finfo;
    apr_status_t apr_err;
    apr_int32_t flags = APR_FINFO_TYPE | APR_FINFO_NAME;
```



```
apr_err = apr_dir_open (dir, directory, pool);
if (apr_err)
    return apr_err;

/* Loop over the directory entries, clearing the subpool at the top of
   each iteration.  */
for (apr_err = apr_dir_read (finfo, flags, dir);
     apr_err == APR_SUCCESS;
     apr_err = apr_dir_read (finfo, flags, dir))
{
    const char *child_path;

    /* Skip entries for "this dir" ('. ') and its parent ('..').  */
    if (finfo.filetype == APR_DIR)
    {
        if (finfo.name[0] == '.'
            (finfo.name[1] == 'W0'
             || (finfo.name[1] == '.' finfo.name[2] == 'W0'))))
            continue;
    }

    /* Build CHILD_PATH from DIRECTORY and FINFO.name.  */
    child_path = svn_path_join (directory, finfo.name, subpool);

    /* Do some task to this encountered path.  */
    do_some_task (child_path, subpool);

    /* Handle subdirectories by recursing into them, passing SUBPOOL
       as the pool for temporary allocations.  */
    if (finfo.filetype == APR_DIR)
    {
        apr_err = crawl_dir (files, child_path, subpool);
        if (apr_err)
            return apr_err;
    }

    /* Handle files by adding their paths to the FILES array.  */
    else if (finfo.filetype == APR_REG)
    {
        /* Copy the file's path into the FILES array's pool.  */
        child_path = apr_pstrdup (hash_pool, child_path);

        /* Add the path to the array.  */
        (*((const char **) apr_array_push (files))) = child_path;
    }

    /* Clear the per-iteration SUBPOOL.  */
    svn_pool_clear (subpool);
}

/* Check that the loop exited cleanly.  */
if (apr_err)
    return apr_err;

/* Yes, it exited cleanly, so close the dir.  */
apr_err = apr_dir_close (dir);
if (apr_err)
    return apr_err;

/* Destroy SUBPOOL.  */
svn_pool_destroy (subpool);

return APR_SUCCESS;
}
```

이 예는 루프와 재귀적인 상황의 **양쪽 모두에서의** 효율적인 풀의 이용법을 설명하는 것입니다. 각각의 재귀는 함수에 건네주는 풀의 서브 풀을 만드는 것으로 시작됩니다. 이 풀은 루프의 area로 이용되어 각각의 반복으로 클리어 됩니다. 이 결과, 메모리의 이용은, 대략적으로 말해 재귀의 깊이에만 비례해, 최상정도 디렉토리의 아이로서의 파일과 디렉토리의 합계수에는 비례하지 않습니다. 이 재귀 함수의 최초의 호출이 종료한 시점에서, 건네준 풀에 보존된 데이터는 실제로는 매우 작은 것이 됩니다. 이 함수가, `alloc()` 와 `free()` 함수를 하나 하나의 데이터에 대해서 호출하지 않으면 안 된다고 했을 때의 복잡함을 생각해 보세요!

풀은 모든 어플리케이션에 이상적인 것은 아닐지도 모릅니다만 Subversion에서는 매우 태우는데 좋습니다. Subversion 개발자로서 풀의 이용에 친해져, 어떻게 그것을 올바르게 사용할까에 정통하지 않으면 안 됩니다. 메모리 이용에 관계한 버그와 메모리 리크는 API의 종류에 의하지 않고, 진단해, 수정하는 것은 어려운 것입니다만, APR 에 의해 준비된 풀의 작성은, 매우 편리해, 시간의 절약으로 연결되는 기능을 가지고 있습니다.

1.6. Subversion에의 공헌

Subversion 프로젝트에 대한 정보의 공식적인 문서는 물론, 프로젝트 웹 사이트의 <http://subversion.tigris.org>입니다. 거기에 원시 코드에 액세스 하는 방법이나, 메일링 리스트에 참가한다 방법에 대한 정보가 있습니다. Subversion 커뮤니티는 언제라도 새로운 참가자를 환영하고 있습니다. 만약 원시 코드를 변경한다고 하는 형태의 공헌에 의해 이 커뮤니티에 참가하는 것에 흥미가 있다면, 어떤 느낌에 시작하면(자) 좋은가의 힌트를 둡니다.

1.6.1. 커뮤니티에의 참가

커뮤니티에 참가하는 최초의 스텝은 최신의 정보를 언제라도 입수할 수 있다 방법을 찾아내는 것입니다. 이것을 제일 효율적으로 하려면, 개발자의 논의를 위한 메일링 리스트에 참가해(<dev@subversion.tigris.org>), 커밋 메일링 리스트에 참가하는 것입니다(<svn@subversion.tigris.org>). 이러한 메일링 리스트에 있는 정도 대략적으로 따라가는 것만으로도, 중요한 디자인상의 논의에 액세스 할 수 있고, Subversion 원시 코드에 의 실제의 수정을 볼 수가 있고, 이러한 변경의 상세한 리뷰에 입회해, 변경을 제안할 수가 있습니다. 이러한 e-mail 베이스의 논의의 장소는 Subversion 개발에서의 최대 중요한 커뮤니케이션 수단입니다. 다른 흥미가 있는 Subversion 관련 리스트에 대해서는, Web 사이트의 메일링리스트의 섹션을 봐 주세요.

그러나, 무엇이 필요한가라는 것을 어떻게 알면 좋은 것일까요? 프로그래머에 있어, 개발을 돕자고 하는 큰 의도를 가져 있지만, 좋으면 담당자를 잡을 수 없는 것은 자주 있는 것입니다. 결국, 굵고 싶다고 생각할까 해 장소가 어딘가를 벌써 알고 있어 커뮤니티에 참가하는 사람은 그만큼 많지는 않습니다. 그러나, 개발자의 논의를 뒤쫓는 것에 의해, 벌써 존재하고 있는 버그나, 난무하는 기능 요구에 주의를 향할 수가 있어, 그 어떤 것인가가 당신의 흥미를 당길지도 지선. 또, 미해결의, 할당이 정해져 있지 않은 작업을 찾는 좋은 장소와 해, Subversion 웹 사이트상의 Issue Tracking 데이터베이스가 있습니다. 거기서 현시점에서 벌써 알려져 있는 버그와 기능 요구의 일람을 보는 것이 할 수 있습니다. 만약 무엇인가 작은 일로부터 시작하고 싶다면, "bite-sized" 그렇다고 하는 표가 붙은 문제를 봐 주세요.

1.6.2. 원시 코드의 취득

코드를 편집하려면, 우선은 코드를 손에 넣을 필요가 있습니다. 이것은 공개의 Subversion 소스 저장소(repository)로부터 작업 카피를 체크아웃 하지 않으면 안 되는 것을 의미합니다. 간단하게 들립니다만, 약간 기교적인 작업에 됩니다. Subversion의 원시 코드는, Subversion 자신에 의해 버전 관리 되고 있으므로, 무엇인가 다른 방법으로 벌써 동작하는 Subversion을 취득하는 것에 의해 "최초의 단서를 얻을"필요가 있습니다. 제일 보통 방법은, 최신의 바이너리 패키지를 다운로드한다(당신의 머신으로 이용할 수 있는 것이 있는 경우입니다만), 인가, 최신의 원시 코드의 tarball (을)를 다운로드해, 자신의 Subversion 클라이언트를 만들까입니다. 만약 소스로부터 생성하는 것일 수 있는은, 순서에 대해서는 소스 트리의 최상정도에 있는 INSTALL 파일에 반드시 대충 훑어봐 주세요.

동작하는 Subversion 클라이언트를 손에 넣으면,
<http://svn.collab.net/repos/svn/trunk> 에 있는 Subversion의 소스 저장소

(repository)의 작업 카피를 체크아웃 할 준비 하지만 되어 있습니다: [\[7\]](#)

```
$ svn checkout http://svn.collab.net/repos/svn/trunk subversion
A HACKING
A INSTALL
A README
A autogen.sh
A build.conf
...
```

위의 커멘드는, 최첨단의, 최신 버전의 Subversion의 원시 코드 (을)를, 현재의 작업 디렉토리에 subversion 라고 한다 이름의 서브 디렉토리를 만들어 체크아웃 합니다. 분명하게, 마지막 인수는, 개별의 환경에 응해 조정할 수가 있습니다. 새로운 작업 카피 디렉토리를 어느 게 부르려고, 이 조작이 완료하면 Subversion의 원시 코드를 취득 되어 있습니다. 물론, 그 밖에도 몇개의 보조적인 프로그램 라이브러리가 필요하게 됩니다 (apr, apr-util, 등 등) 자세한 것은 작업 카피의 최상정도에 있는 INSTALL 파일을 봐주세요.

1.6.3. 커뮤니티의 방식에 정통하는 것

이것으로 Subversion 원시 코드의 최신판이 있는 작업 카피를 손에 넣었다 의로, 아마 작업 카피의 최상정도 디렉토리에 있는 HACKING 파일을 보면서 디렉토리안을 이것저것 조사하고 싶다고 생각하겠지요. HACKING 파일은 Subversion에 공헌하기 위한 일반적인 수속이 포함되어 있어, 거기에는 어떻게 해, 나머지의 코드와 모순되지 않는 형태로 당신의 원시 코드를 올바르게 쓰는지라든가, 제안하고 싶은 변경점에 어떠한 효율적인 변경 로그 메시지를 적는지, 어떻게 변경점을 테스트하면 좋은지, 등이 포함됩니다. Subversion의 소스 저장소(repository)에 대한 커밋 권한은 획득하지 않으면 안 됩니다. 실력 본위의 정부에 의해, [\[8\]](#) HACKING 파일은 자신이 제안하려 하고 있는 변경이 기술적으로 거부되는 것 없게 칭찬에 적합한지 어떤지를 확인하기 위해서는 이 이상 없고 귀중한 자료입니다.

1.6.4. 코드의 변경과 테스트

코드와 커뮤니티의 폴리시를 이해하면, 변경에 착수하는 것이 할 수 있습니다. 큰 문제에 임하고 있는 경우에서도, 거대한, 전부 기존의 것과 바꾸어 버리는 것 같은 수정을 하는 대신에, 작은, 그러나 관련의 어느 변경의 모임을 만들려고 하는 것이 최선의 방법입니다. 하려 하고 있는 것에 필요한 코드의 수정을 할 수 있는 한 적으면, 제안하자(으)로 하고 있는 변경은 그 만큼 간단하게 이해되겠지요(그리고, 검토하는 것도 편해 깊어진다). 수정세트의 각각을 베푼 후에는, 당신의 Subversion 트리는 컴파일러가 경고를 하나도 내지 않는 상태가 되어 있어야 합니다.

Subversion에는 꽤 철저히 했다 [\[9\]](#) 데그레이트를 체크하기 위한 테스트 스위트가 있어, 제안하려 하고 있는 변경은, 어떠한 테스트에서도 실패하지 않게 되어 있는 것이 바람직합니다. 소스 트리의 최상정도로 **make check** (을)를 실행하는(Unix의 경우) 일로, 자신의 변경의 체크를 할 수가 있습니다. 당신의 공헌이 거절되는 제일 빠른 방법은(적절한 로그 메시지를 적지 않았다 경우 이외는), 테스트가 통과하지 않는 변경을 보내는 것입니다.

제일 좋은 시나리오, 실제로 적절한 테스트를, 테스트 스위트하게 추가해, 그래서 당신의 변경점이 실제로 기대한 것처럼 동작하는 것입니다. 실제조사 있고, 가끔 사람이 공헌할 수 있는 최선은 새로운 테스트를 단지 추가 하는 것입니다. 에러의 계기가 되는 것 같은 향후의 수정으로부터 지키는 것 같은 의미를 담아, 현재의 Subversion로 동작하고 있는 기능을 위해서(때문에) 데그레이드의 테스트를 쓸 수가 있습니다. 또, 벌써 알려져 있는 실패를 보이기 위한 새로운 테스트를 쓰는 일도로 옵니다. 이 목적을 위해서(때문에)는 Subversion 테스트 스위트는, 어느 테스트는 실패 하는 것이 기대되고 있는 것이라고 지정하는 것을 인정합니다. (XFAIL라고 합니다), 그리고 Subversion가 기대하는 형태로 실패하는 한, 그 테스트의 결과인 XFAIL 자체는, 성공했다고 보입니다. 마지막으로, 좋은 테스트 스위트를 준비하면 할 뿐(만큼), 이해하기 어려운 데그레이트의 버그 (을)를 진단하기 위해서 낭비되는 시간을 줄일 수가 있습니다.

1.6.5. 변경점의 제공

원시 코드에 대한 수정을 한 후, 명료해 결정된 로그 메시지를 만들어, 그러한 변경을 설명해, 그 이유를 써 주세요. 그리고 email를 개발자용 메일링리스트에 보내, 거기에는 로그 메시지와 **svn diff**의 출력(이것은 Subversion의 최상정도 작업 카피 그리고 실행해 주세요)를 포함해 주세요. 커뮤니티의 멤버가 당신의 변경이 받아들여진다고 판단했을 경우, 커밋 권한을 가졌다 누군가(Subversion의 소스 저장소(repository)에 새로운 리비전을 만드는 허가를 가지고 있는 사람)이, 당신의 변경이 공개된 원시 코드 트리에 추가합니다. 저장소(repository)에 대해서 변경을 직접 커밋하는 권한은, 이점이 있는 경우에만 인정됩니다만약 Subversion의 이해나, 프로그래밍의 능력이나,"팀 스피리트"를 나타내면, 당신은 반드시 그 권한을 얻을 수 있겠지요.

Notes

[1]

Berkeley DB 의 선택은, Subversion가 필요로 하고 있는 몇개의 자동적인 기능이 있습니다. 그것은, 데이터 일관성, 불가분의 기입 기능, 복구 가능성, 온라인 백업, 등입니다.

[2]

우리는, 시간은 실제로는**제 4** 차원이다 그렇다고 하는 인상을 쪽 가지고 있던 SF 파일에 쇼크를 준다고 하는 것을 이해하고 있고, 다른 이론을 우리가 선언하는 것에 의해 생긴다 심리적인 트라우마에 대해서는 사과하지 않으면 안됩니다.

[3]

Subversion 는 ANSI 시스템 콜과 데이터형을 할 수 있는 한 이용하고 있습니다.

[4]

Neon 와 Berkeley DB 는 그러한 프로그램 라이브러리의 예입니다.

[5]

즉, 엔트리의 URL는 친디렉토리 URL와 엔트리 명칭을 연결한 것과 같은으로 간주한다고 하는 것입니다.

[6]

이해와 같이, 어쨌든 Subversion 는 좀 더 최근이 되어, 고유의 프로토콜에 진화했습니다. libsvn_ra_svn 모듈의 실장이 이것입니다.

[7]

이 예로 체크아웃 하는 URL는,svn로 끝나는 것은 없고,trunk라고 하는 서브 디렉토리가 되어 있습니다. 이 이유에 대해서는, Subversion의 브랜치(branch)와 태그 모델의 논의를 참조해 주세요.

[8]

이것은 무엇인가의 엘리트 주의와 같이 보일지도 모르지 않습니다만,"커밋 권한을 획득한다"라고 하는 개념은 효율을 고려한 일입니다 안전해 도움이 되는 누군가의 변경을 검토해 적용하기 위해(때문에) 노력에 걸리는 시간과 위험한 변경을 바탕으로 되돌린다고 하는 잠재적인 시간과의 사이의 균형입니다.

[9]

아마, 팝콘이라도 먹고 싶어질지도. 여기서의"철저한"은, 비대화적인 머신으로, 약 30분 걸린다고 하는 정도의 의미로 번역해 주세요.

[EditText](#) | [Refresh](#) | [FindPage](#) | [DeletePage](#) | [LikePages](#) | [Tour](#) | [Keywords](#) |

W3C XHTML 1.0 W3C CSS  POWERED

Last modified: 2005-06-01 14:12:17, Processing time: 0.0447 sec

[일정관리프로그램](#)

Google에서 전사용 공유 캘린더를 무료로 제공. 지금 다운로드 하세요!

[KTNET 중소기업용 웹ERP](#)

회계/재고/영업/급여/생산/그룹웨어 저렴한 월 사용료 4만원으로 즉시 이용